

ANTERN STUDY

CONTENTS

AN ARCHITECTURE STUDY

Designing an AI Pull-Request Review Agent

A PR review agent is not a linter with an LLM bolted on. It is a fan-out of specialist reasoners over a diff, grounded in retrieved codebase context, with every action written to one time-ordered spine. This study derives that architecture from first principles — and the pieces it carries forward assemble, by the end, into one defensible design.

Published July 22, 2026 A first-principles derivation

Lenses: design template, agentic failure modes, one-database data spine

Read top to bottom

HOW TO READ THIS. The pages that follow do not start with boxes and arrows. They start with questions — why an automated reviewer exists at all, how a senior engineer actually reviews, what kinds of knowledge a review needs. Each step in the climb ends with two things: **the question we ask** (a reusable thinking move that generalizes to other systems) and **a piece we carry into the architecture** (a concrete component, field, or rule). By the end those carried pieces assemble into the whole design, and every box can be traced back to the question that justified it.

Treat every section as a claim to interrogate, not a conclusion to copy. Where would this break at ten thousand pull requests per hour? Which assumption is doing the quiet

work? What would you have decided differently? Write the question in the margin and keep going.

PART 0 / THE THINKING MINDSET

The lens before the system

Before any code-review-specific design, four reusable instruments. A template that turns any messy workflow into an agentic system. A catalog of the ways agentic systems fail. A spectrum for deciding how much human stays in the loop. And a lifecycle that scaffolds the whole build. None of these are about PR review yet. They are the way of seeing that the rest of the study uses.

How do you design an agent that reviews code as well as a careful senior engineer – and proves its work?

THE DRIVING QUESTION

0.1 The Universal Design Template

After enough systems, intuition becomes a repeatable process. Five moves turn any workflow, in any domain, into an agentic design. We will apply all five to PR review in Part I.

Move 1. Map the mess

Document what actually happens today, not the process document nobody follows. Watch the person doing the work. Record the trigger that starts it, every step in the middle, the deliverable at the end, where the human is actually *thinking* versus mechanically shuffling information, and where it breaks.

Move 2. Name the trigger and the output

Every automatable workflow has a precise trigger and a precise output. "A claim email arrives at claims@company.com," not "claims come in." "A structured review is posted to the PR," not "code gets reviewed." If you cannot state both in one sentence each, you have not looked closely enough.

Move 3. Assign components

Take each step and assign it to a component type. Detecting that work is needed is a **trigger**. Fetching data is a **tool/API**. Reading unstructured input or writing language is an **LLM**. A score or classification that must be identical every time is **deterministic ML**. A judgment with legal, financial, or safety stakes is a **human checkpoint**.

MOST COMMON MISTAKE

Assigning an LLM to a step that should be deterministic. If the output must be the same every time given the same input – a severity score, a routing class – it must be deterministic. If the output is language and some variation is acceptable, it can be an LLM.

Move 4. Choose autonomy

Decide how much the system does on its own versus how much it defers to a human. This is not a default – it is a design choice driven by the consequence of error. Section 0.3 gives the full spectrum.

Move 5. Design for failure

Walk each component and break it on purpose. What happens when the LLM hallucinates, the API times out, two parallel agents conflict? Every component should fail gracefully.

The system should degrade to slower but correct, never fast but wrong. The worst failure mode is a wrong answer delivered with confidence.

0.2 The Failure Modes Catalog

Every agentic system fails. The only question is whether it fails safely. Seven recurring modes, and how to design against each. We will run this catalog against code review in L8.

SEVEN GENERAL AGENTIC FAILURE MODES

MODE	WHAT HAPPENS	DESIGN AGAINST IT WITH
Hallucination in a critical path	The model states something plausible but false in a place that matters	Citation requirement, a fact-check layer, human review for high stakes, prompts that permit "I don't know"
Model drift	Accurate at deployment, degrades as the world changes	Monitoring dashboard, alert thresholds, periodic retraining, rules fallback
Tool / API timeout	An external system stalls and the pipeline hangs	Timeout-and-retry, graceful degradation on partial data, circuit breaker on a dead service
Feedback-loop poisoning	Bad feedback is stored and degrades future behavior	Minimum evidence threshold, audit for protected proxies, decay on old feedback, human reset
Orchestration deadlock	Two parallel steps wait on each other; a merge never receives an input	Timeouts on every step, health checks, idempotent operations, dead-letter queue
Human bottleneck	Auto-handling works, but the escalation queue grows faster than humans clear it	Escalation-rate monitoring, queue prioritization, threshold tuning under load, honest capacity planning
The "almost-right" problem	Output 90% correct, 10% subtly wrong, while reviewers drift into complacency	Rotate reviewers, flag low-confidence outputs, random audits, inject known-wrong inputs to test vigilance

0.3 The Human-in-the-Loop Spectrum

Not every system needs the same level of human involvement. The right level is a design choice, not a default. Five levels, and the three factors that pick one.

LEVEL	DESCRIPTION	WHERE IT FITS
Full automation	System handles everything; human samples periodically	Routine, low-stakes, reversible work
Human reviews output	System produces, human verifies before it goes out	Drafts with reputational stakes
Human handles exceptions	System auto-handles easy cases, human sees the hard ones	Anomalies, low-confidence cases, escalations
Human decides, system prepares	System gathers all context, human makes the call	High-consequence, irreversible decisions
Full human with AI assist	Human does the work, system helps at specific steps	Early-stage, low-trust, or creative work

Three factors choose the level. **Consequence of error:** a wrong style comment is annoying; a missed SQL injection is dangerous. **Reversibility:** an auto-posted review can be disputed and removed; a merged migration cannot be un-run easily. **System maturity:** new systems need more oversight; proven ones earn less.

Start with more human involvement than you think you need. Reduce it as the system proves itself. It is far easier to remove a checkpoint than to recover from removing it too early.

0.4 The 20-Phase Production Lens

Beginners design the happy path: receive, reason, respond. Senior engineers add the back half: observe, secure, recover, govern, optimize, learn. A twenty-phase lifecycle names every phase, and our build plan in Part IV follows it exactly.

THE PRODUCTION LIFECYCLE AS A THINKING SCAFFOLD

PHASE	CONCERN
-------	---------

PHASE	CONCERN
0 Cognitive Design	What thinking should the system perform; autonomy level; HITL boundaries
1 System Architecture	Boundaries, style, module graph, ADRs
2 Frontend	Dashboard shell, streaming, agent transparency
3 Backend & API	FastAPI, webhook, idempotency, state
4 Workflow Orchestration	Topology, checkpointing, parallel fan-out
5 LLM & Reasoning	Model routing, structured output, prompt registry
6 Memory Architecture	RAG, hybrid retrieval, the vector lane
7 Tooling & Sandboxing	Tool registry, permissions, Docker isolation
8 Multi-Agent Systems	Roles, contracts, the aggregator
9 Evaluation	Golden datasets, LLM-as-judge, regression gates
10 Observability	Traces, token cost, alerts – the events spine
11 Security	Threat model, prompt injection, RBAC, Zero Trust
12 Reliability	Circuit breakers, idempotency, checkpointing
13 Infrastructure	Containers, queues, data-layer provisioning
14 Data Engineering	Ingestion pipelines, schema, encoding
15 Governance	Audit trails, explainability, residency
16 Economics	Token cost attribution, budget caps, routing efficiency
17 Developer Experience	Prompt playground, trace viewer, replay
18 CI/CD for AI	Prompt versioning, eval gates, canary releases

PHASE

CONCERN

19 Human-in-the-Loop Approval workflows, escalation, dispute

20 Continuous Learning Feedback loops, drift detection

PART 0 – CARRY FORWARD

Four reusable instruments, none specific to code review yet:

- A **five-move design template**: map the mess, name trigger & output, assign components, choose autonomy, design for failure.
- A **seven-mode failure catalog** to run against any agentic design.
- A **five-level HITL spectrum**, chosen by consequence, reversibility, and maturity.
- A **twenty-phase lifecycle** that scaffolds the build and forces the back half into view.

PART I / FIRST PRINCIPLES

Why automated review exists, before how it works

Most people ask how to build a PR review agent. The better question is why one exists at all, because once you can answer that, the architecture becomes almost obvious. This part climbs from that first principle to a complete reasoning model in ten levels.

Read it twice. The first time for the review agent. The second time for the method – the order of questions a strong engineer asks when handed any system they have never seen. Each level ends with **the question we ask** (the reusable move) and **carry into the architecture** (the concrete piece we keep). By the end those pieces assemble into one design.

A developer opens a pull request. A webhook fires. What, exactly, should happen next – and why?

L0 Why This System Exists

Imagine a team without automated review. Every pull request waits on a senior engineer's attention. That attention is the scarce resource: it is slow (PRs queue for hours or days), inconsistent (the same issue is caught on Monday and missed on Friday), and fatigued (the tenth review of the day is not the first). The cost is not "no review happens" — it is that the most expensive, most valuable human time on the team is spent on work that is, in large part, mechanical pattern recognition.

So an automated reviewer exists to solve exactly one problem: **reclaiming senior-reviewer attention by automating the mechanical part of review**, so the human is spent only where judgment is genuinely required.

A review agent is not a replacement for human judgment. It is a way to spend human judgment only where it is actually scarce — and to do the rest consistently, tirelessly, and at any hour.

Hold onto the word *selective*. The system should not flood the PR with every conceivable comment. It should surface high-value findings and route uncertain ones to a human. That selectivity is the seed of the whole architecture.

THE QUESTION WE ASK

Why does this system exist at all, and what single cost or loss does it remove?

Generalizes to: before any component, name the one scarcity the system relieves. A search system relieves the cost of not finding; a payments ledger relieves the cost of disputed truth. If you cannot name it, you are designing a feature nobody needs.

CARRY INTO THE ARCHITECTURE

A selective, high-value posture. The system optimizes for surfacing findings worth a senior's attention and deferring the rest – not for maximal output. Selectivity, not coverage, is the first principle.

L1 Start From How a Senior Reviews

Before touching LLMs, prompts, or vector databases, ask how a senior engineer actually reviews a pull request. When you are designing something genuinely hard, look for a system that already solved it. Human review has been refined over decades, and the structure we need is sitting inside how a good reviewer thinks.

Watch one closely. They do four things that a naive single-prompt reviewer does not. They **bring codebase context** – they know this function overrides a base class, that this pattern contradicts a past decision. They **reason across separate concerns** – a security pass, a correctness pass, a test-coverage pass, a documentation pass, each with a different mindset. They **stay skeptical** – they do not assume the diff is correct. And they **cite evidence** – "this is wrong because line 40 can be null here," not "looks off."

Read those four again as an engineer. "Brings codebase context" means the agent needs retrieval. "Reasons across separate concerns" means the agent is not one reasoner but several. "Stays skeptical" and "cites evidence" mean every finding needs a rationale and a confidence. The way a senior reviews is not trivia – it is a decomposition waiting to be built.

Security concern / "could this be exploited?"

Injection risks, secrets in code, auth bypasses, unsafe deserialization. A distinct mindset from correctness.

Quality concern / "is the logic right?"

Correctness bugs, logic errors, code smells, unnecessary complexity. The classic review pass.

Tests concern / "what's untested?"

Missing cases, untested edge conditions, brittle assertions, coverage gaps.

Docs concern / "will the next reader understand?"

Missing docstrings, outdated comments, undocumented public APIs, decisions left unexplained.

THE QUESTION WE ASK

Is there a mature system – human or engineered – that already solved a version of this, and what structure can I borrow from it?

Generalizes to: analogies hand you a ready-made decomposition. A recommendation system borrows from word of mouth; a cache borrows from working memory; a review agent borrows from how an expert reads.

CARRY INTO THE ARCHITECTURE

Four specialist concerns, born from how a human reviews. The system is not one reasoner but four – security , quality , tests , docs – each a distinct pass with its own mindset. This is the seed of the multi-agent design.

L2 Map the Mess

Now apply the Part 0 template. **Map the mess** of what happens on a PR today: a developer pushes a commit, opens a PR, and then waits. Eventually a reviewer notices, context-switches into the change, reads the diff, sometimes pulls the branch to run it, leaves comments, and the developer iterates. The waiting and the context-switching are pure cost.

Name the trigger and the output. The trigger is precise: GitHub emits a `pull_request` webhook when a PR is opened or updated. The output is precise too: a single structured review, posted back to that PR, with findings attached to specific files and lines.

That word *structured* is doing strategic work. A review is not a blob of prose. It is a list of findings, each with a shape: which concern raised it, how severe, where in the code, why, and how sure the agent is. Deciding that shape now is what lets every later

component – the aggregator, the human-review gate, the audit trail – do its job. A first cut of that shape:

`agent_type` / which concern raised this

security, quality, tests, or docs – so findings can be grouped and attributed.

`severity + category` / how bad, what kind

CRITICAL down to INFO; a category like "injection" or "missing-test."

`file / line` / the exact location

So the finding posts inline on the PR, not as a vague summary.

`confidence + rationale` / how sure, and why

Confidence drives the human-review gate. Rationale is what makes the finding auditable and disputable.

THE QUESTION WE ASK

What is the precise trigger, the precise output, and the shape of the object that travels between every component?

Generalizes to: a system is components plus the contract between them. Name the trigger and the output first, then name the object on the arrows. The object usually matters more than the boxes.

CARRY INTO THE ARCHITECTURE

An ingress trigger and a structured findings contract. The trigger is a GitHub webhook; the output is a structured review. The unit that flows through the system is a **Finding** with `agent_type` , `severity` , `category` , `file / line` , `confidence` , and `rationale` .

L3 Industry-Standard Thinking

Most people picture one arrow: diff in, LLM, comments out. That is not how a capable review system works, and the gap between those pictures is the gap between a demo and a product. Walk the rungs of how the industry has climbed toward review automation, and watch why each prior rung falls short.

FOUR RUNGS OF REVIEW AUTOMATION

RUNG	WHAT IT DOES	WHY IT FALLS SHORT
Linters	Pattern-match syntax and style rules	No semantics. Cannot reason about intent, logic, or whether a test is meaningful.
Static analysis	Data-flow and type analysis; finds some real bugs	High false-positive rate; no codebase-wide judgment; cannot read documentation intent.
Single-LLM review	One prompt judges the whole diff	One mindset for four concerns; no grounding in the repo; hallucinates with confidence; no audit.
Agentic fan-out	Specialist agents, each grounded, each skeptical, merged by an aggregator	The rung this design stands on – but it demands orchestration, retrieval, and a proof layer.

The single-LLM rung is the seductive one, because it works in a demo. It collapses four concerns into one prompt, which means it does each of them shallowly, and it has no way to be grounded, audited, or trusted. The fan-out rung – running the four concerns from L1 as parallel specialists – is where production lives.

THE QUESTION WE ASK

What does the mature version of this decompose into, beyond the happy path the demo shows?

Generalizes to: every interesting system has a hidden back half. List the rungs the industry has already climbed, and stand on the highest one whose cost you can actually pay.

CARRY INTO THE ARCHITECTURE

Parallel specialists, not a single prompt. The four concerns from L1 run as four agents in parallel, each doing one job deeply. This is the agentic fan-out, and it implies an

orchestrator and an aggregator we will name later.

L4 The Grounding Problem

Here is where most single-LLM reviewers fail. An LLM handed a diff in isolation knows what changed but not what it changed *within*. It cannot know that this function overrides a base method, that this pattern was deliberately rejected in a past decision, or that the test convention in this repo is table-driven. So it does what models do under uncertainty: it guesses, confidently. That is hallucination in a critical path – the first failure mode from the 0.2 catalog.

A senior reviewer does not have this problem because they *know the repo*. The design question is: what gives the agent that knowledge? It cannot be the full repository in the prompt – that exhausts the context window on any non-trivial codebase, and most of it is irrelevant to a given diff. The answer is retrieval: for each diff, fetch only the most relevant slices of the codebase and put those in the prompt.

An ungrounded reviewer is a confident stranger. A grounded one is a colleague who has read the code. Retrieval is what turns the stranger into the colleague.

THE QUESTION WE ASK

What does this reasoner need to know that is not in front of it, and how do we put exactly that – and only that – in front of it?

Generalizes to: any LLM judging an artifact in isolation hallucinates. Retrieval-augmented generation is the general fix – fetch the relevant context, do not dump everything, do not assume the model already knows.

CARRY INTO THE ARCHITECTURE

A retrieval layer (RAG). Each specialist queries for the codebase context relevant to the diff and reasons over diff-plus-context, never the diff alone. Grounding is not optional; it is what separates this from a single-LLM reviewer.

L5

What Kinds of Memory Does Review Need?

L4 said "retrieve context." But context is not one kind of thing. A senior reviewer draws on several kinds of memory, and – exactly as in cognitive science – each kind has a different shape and a different access pattern. Naming them now is what determines the data design later.

KIND OF MEMORY	WHAT IT HOLDS FOR REVIEW	THE DATA SHAPE IT WANTS
Semantic	The codebase itself – functions, classes, modules, ADRs, conventions, as meaning	Vector embeddings + similarity search
Episodic	Past reviews – what was flagged before, what was disputed, what was merged	Time-stamped relational rows
Procedural	How this team likes things done – conventions, ADRs, severity policy	Small, high-priority, almost always loaded

Read those three again as an engineer. Semantic memory wants similarity search over embeddings – a vector store. Episodic memory wants time-ordered, queryable rows – relational history. Procedural memory is small and structured – facts and rules. The taxonomy is not philosophy; it is a schema waiting to be written, and it tells us we do not have one data need but three distinct shapes.

THE QUESTION WE ASK

What distinct kinds of state does this system hold, and does each kind want a different shape rather than one undifferentiated bucket?

Generalizes to: before choosing storage, enumerate the kinds of state by access pattern. Hot vs cold, similarity vs exact, append-only vs mutable. The kinds drive the shapes; the shapes drive the stores.

CARRY INTO THE ARCHITECTURE

Three data shapes. Semantic memory of the repo wants a **vector** / **ANN** shape. Past reviews and findings want a **relational** shape. Conventions and decisions want a small

structured shape. Hold these three – Part II decides how many actual databases they require.

L6 Trust and Proof

Suppose the agent posts a finding: "this endpoint is vulnerable to SQL injection, confidence 0.6." A developer disputes it. Now what? If there is no record of *why* the finding was raised – which context was retrieved, which prompt version ran, what the model returned, what it cost – the system cannot defend itself, cannot be debugged, and cannot improve. A review you cannot audit is worthless, and a system whose cost can run away unobserved is dangerous.

So trust requires a third thing beyond reasoning and grounding: **proof**. Every action the agent takes – every span of work, every LLM call, every tool call, every decision – must be recorded as an event, in time order, durably. That single stream of events is what powers three things at once: a trace viewer (reconstruct any review end-to-end), an audit trail (defend or dispute any finding), and a cost ledger (attribute every token to an agent and a model).

If the system cannot show its work, it has not done the work. The proof layer is not instrumentation bolted on at the end – it is born here, as a first principle, the moment we ask the system to be trusted.

THE QUESTION WE ASK

When this system produces an output, can it prove how it got there – and can it tell us what that cost?

Generalizes to: any system making consequential automated decisions needs an immutable, time-ordered record of its actions. Payments, fraud scoring, medical triage. Design the audit stream with the decision, not after the incident.

CARRY INTO THE ARCHITECTURE

An events spine. Every action becomes a time-ordered event row – span, LLM call, tool call, decision – carrying cost, latency, confidence, and outcome. One stream feeds the trace viewer, the audit trail, and the cost ledger. This is a fourth data need, time-series in shape.

L7 **When Not to Trust It**

L0 said the system should be selective. L6 gave it the confidence field. Now apply the 0.3 HITL spectrum to *this* system. The agent is not always right, and it knows roughly when it is unsure – that is what the `confidence` on each finding measures. The design decision is what to do with that knowledge.

The answer is a confidence-weighted gate. High-confidence reviews, with no critical findings, post automatically – the agent has earned that autonomy. Low-confidence reviews route to a human approval queue. Any finding marked `CRITICAL` escalates regardless of confidence, because the consequence of a missed critical issue is too high to automate (consequence of error, from 0.3). This places the system at the "human handles exceptions" level of the spectrum, with an escalation path to "human decides."

THE CONFIDENCE-WEIGHTED GATE

CONDITION	ACTION	WHICH 0.3 FACTOR
High confidence, no CRITICAL	Post automatically	Maturity earns autonomy
Confidence below threshold	Route to human approval queue	Uncertainty, defer judgment
Any CRITICAL finding	Escalate, page a human	Consequence of error too high
Developer disputes a posted finding	Route to dispute, record feedback	Reversibility, learning loop

THE QUESTION WE ASK

Where on the human-involvement spectrum does this system belong, and what signal moves a given case up or down it?

Generalizes to: autonomy is not binary. Let the system earn it case by case, gated on a confidence signal and the stakes. Content moderation, lending, triage – all use the same confidence-weighted routing.

CARRY INTO THE ARCHITECTURE

A confidence-weighted HITL gate. The aggregator computes an overall confidence; below threshold or on any CRITICAL finding, the review enters a human approval queue instead of posting. This implies queue and feedback tables in the relational shape from L5.

L8 Failure Modes, Applied

Run the 0.2 catalog directly against code review. Each general failure mode lands on a specific defense, and – notice – each defense is a component we have already carried or are about to add.

GENERAL FAILURE (0.2)	IN CODE REVIEW IT LOOKS LIKE	DEFENSE
Hallucination in critical path	A finding about code the agent never actually saw	Grounding (L4) + rationale + confidence (L2)
Tool / API timeout	The LLM provider or GitHub API stalls	Retries with backoff, circuit breakers
Orchestration deadlock	The aggregator waits forever on a hung agent	Timeouts on every node, dead-letter handling
The "almost-right" problem	A finding 90% right but subtly misattributed	Dedup across agents, confidence threshold, HITL (L7)
Human bottleneck	The approval queue grows faster than reviewers clear it	Escalation-rate monitoring on the events spine (L6)
Feedback-loop poisoning	The agent "learns" a wrong preference from a few disputes	Minimum evidence threshold before acting on feedback

GENERAL FAILURE

(0.2)	IN CODE REVIEW IT LOOKS LIKE	DEFENSE
Idempotency gap	A retried webhook posts the same review twice	Idempotency key at ingress, dedup before posting

Failure analysis did not invent new parts. It justified the parts we chose and added a thin layer of reliability mechanics on top.

THE QUESTION WE ASK

For each component, what happens when it fails, and does the system degrade to slower-but-correct rather than fast-but-wrong?

Generalizes to: run a pre-mortem before code. Walk each box and break it on purpose. The fallbacks you design now are the incidents you avoid later.

CARRY INTO THE ARCHITECTURE

A reliability layer. Retries, circuit breakers, timeouts, idempotency at ingress, and dedup at the aggregator – each mapped to a specific failure mode, not added speculatively.

L9 The Mental Model

Before we draw a single box, the assembled reasoning in prose. The pieces we carried, in the order they were earned, already describe the whole system.

A pull request triggers the work (L2). It is enqueued, not handled inline, because the trigger must be acknowledged fast and the work decoupled from the ingress (L2, L8). An orchestrator fans the work out to four specialists – security, quality, tests, docs (L1, L3) – running in parallel. Each specialist is grounded by retrieval over the codebase (L4), because an ungrounded reviewer hallucinates. The codebase, the past reviews, and the conventions are three kinds of memory, three data shapes (L5). Each specialist returns structured findings with confidence and rationale (L2). An aggregator merges and deduplicates them, computes an overall confidence, and applies the HITL gate (L7): post automatically when confident, route to humans when not. Every action

along the way is written to an events spine (L6) so the whole thing can be traced, audited, and priced. And a reliability layer (L8) keeps each step degrading to slower-but-correct.

Now ask: how many databases does this need? We have memory, truth, and time. The naive answer is three durable stores. Part II interrogates that answer — and arrives at one.

RUNNING LEDGER / WHAT PART I HAS BUILT

The reasoning model, assembled from first principles

- L0 A **selective, high-value posture**: reclaim scarce senior-reviewer attention.
- L1 The **four specialist concerns**: security, quality, tests, docs — born from how a human reviews.
- L2 An **ingress trigger** and the **Finding contract** with confidence and rationale.
- L3 **Parallel specialists**, not a single prompt — the agentic fan-out.
- L4 A **retrieval layer**: ground every specialist in the codebase.
- L5 **Three data shapes**: vector (semantic), relational (episodic), structured (procedural).
- L6 An **events spine**: every action a time-ordered row, feeding trace + audit + cost.
- L7 A **confidence-weighted HITL gate** routing low-confidence reviews to humans.
- L8 A **reliability layer**: retries, circuit breakers, idempotency, dedup.
- L9 The assembled **mental model** — and the open question: how many databases?

Part II answers the database question. Part III draws the boxes. Part IV lists everything we will build.

PART II / FROM PRINCIPLES TO DATA

Why one database, not three

This is the heart of the data design. The agent needs code memory, durable review truth, and a time-ordered trail of what every agent did. The reflex is to buy one database for each need. This part slows that reflex down and asks a simpler question: can one Postgres-compatible store carry all three shapes without making the system harder to understand?

Most AI systems split memory, truth, and time across separate databases. What if one is enough — and clearer?

THE THESIS OF ADR-003

2.1 Three Data Shapes, One Reflex

Do not start by naming databases. Start by naming what the product has to remember.

The agent has three data shapes. First, **memory**: chunks of code, past reviews, and conventions that help the agent understand a new diff. Second, **truth**: the review row, the findings, the GitHub review ID, and any human decision. Third, **time**: every span, LLM call, tool call, cost, latency, and decision in the order it happened.

There is also one reflex that makes the system feel simple: **read before action, write after action**. Before the agent speaks, fetch relevant code memory. After the agent acts, write the review truth and the event trail.

The naive architecture maps each shape to a purpose-built store. Qdrant for vectors, plain Postgres for review records, and a time-series database for events. Redis still sits alongside for the queue.

THE REFLEXIVE ANSWER: ONE STORE PER SHAPE

DATA SHAPE	REFLEXIVE STORE	WHAT IT HOLDS
Memory	Qdrant	Embedded code chunks for semantic retrieval
Truth	Postgres	Reviews, findings, HITL rows, GitHub IDs
Time	Time-series DB	Spans, LLM calls, tool calls, cost, latency

That sounds clean until you ask one product question: "For this PR, what code did we retrieve, what review did we produce, and which model calls made it expensive?" With three stores, the app has to query three systems and stitch the answer together in Python. That is the cost: more connection pools, more backups, more failure modes, and no simple joins.

THE QUESTION WE ASK

Can we keep the three shapes, but not split them across three durable databases?

Generalizes to: any system tempted to add a new datastore per feature. The right first move is not "which database is popular?" It is "what shape is this data, and does the current store already handle it well enough?"

CARRY INTO THE ARCHITECTURE

Three shapes, one reflex. Memory, truth, and time are requirements. Separate databases are only an implementation choice. The next section asks whether Tiger Cloud lets one Postgres-compatible store carry all three.

2.2 One Store, Not Three

The key sentence is: **Tiger Cloud is Postgres with the missing powers added.**

So this is not "Tiger Cloud versus Postgres" as if they are unrelated. Tiger Cloud gives us a managed Postgres-compatible database, then adds the extensions this agent needs for AI memory and time-series events.

Plain Postgres already handles the truth lane well: review rows, finding rows, foreign keys, transactions, and normal SQL. To absorb the other lanes, it needs three additions.

Each addition has a simple job.

1. Vector search for memory

Imagine every file in the repository gets turned into a small fingerprint made of numbers. Files that mean similar things get similar fingerprints. That number-fingerprint is called an **embedding**. A **vector** is just that list of numbers.

pgvector lets Postgres store that list of numbers in a real column. In this project, that column is `code_chunks.embedding`. So a row does not only say "this is `billing/stripe.py`"; it also stores the numerical meaning of that code.

repo	path	content	embedding
acme/shop	billing/stripe.py	def charge_customer(...)	[0.12, -0.15, ...]
acme/shop	auth/session.py	def refresh_session(...)	[-0.31, 0.05, ...]
acme/shop	tests/payments.py	test_duplicate_charge(...)	[0.10, -0.05, ...]

Now when a new PR adds `charge_customer`, the agent turns the diff into the same kind of number-list and asks: which stored code chunks are closest to this? That is how the agent finds related code without reading the whole repo.

2. pgvector scale and DiskANN

If **pgvector** is the shelf that stores the number-lists, **pgvector scale** is the fast librarian. It adds an index so Postgres can find nearby vectors quickly.

DiskANN sounds complex, but the idea is simple: when there are millions of code chunks, you cannot keep every search shortcut in RAM forever. DiskANN keeps more of the search structure on disk/SSD, and still jumps quickly toward the closest matches. So the agent can search a large code memory without turning the database into a giant RAM bill.

3. Time-series storage for the event trail

A time-series table is a table where time is the main organizing idea. Agent events are exactly like that. Every row has a timestamp: when the security agent started, when the LLM call happened, how much it cost, how long it took, what decision was made.

A normal table stores all those rows together. A **hypertable** stores them as time chunks behind the scenes. To us it still looks like one table, but Tiger can keep Monday's rows, Tuesday's rows, and today's rows in separate chunks internally.

```
agent_events hypertable
```

```
chunk: 2026-07-10
```

```
09:00:01 review_1 security span.start
09:00:04 review_1 security llm.call      cost=0.018
09:00:08 review_1 security span.end
```

```
chunk: 2026-07-11
```

```
10:14:22 review_2 quality span.start
10:14:27 review_2 quality llm.call      cost=0.011
10:14:31 review_2 quality span.end
```

When the dashboard asks for the last hour, Tiger can look at the recent chunk instead of dragging the whole history through the query.

4. Live rollups for dashboards and budget checks

A dashboard should not count raw events from scratch every time it loads. If there are ten million LLM calls, "what did we spend today?" should not scan ten million rows on every refresh.

A **continuous aggregate** is a summary table that Tiger keeps updated for us. Raw rows go into `agent_events` ; Tiger keeps summary rows such as cost per minute, p95 latency, and token totals.

```
raw agent_events rows
```

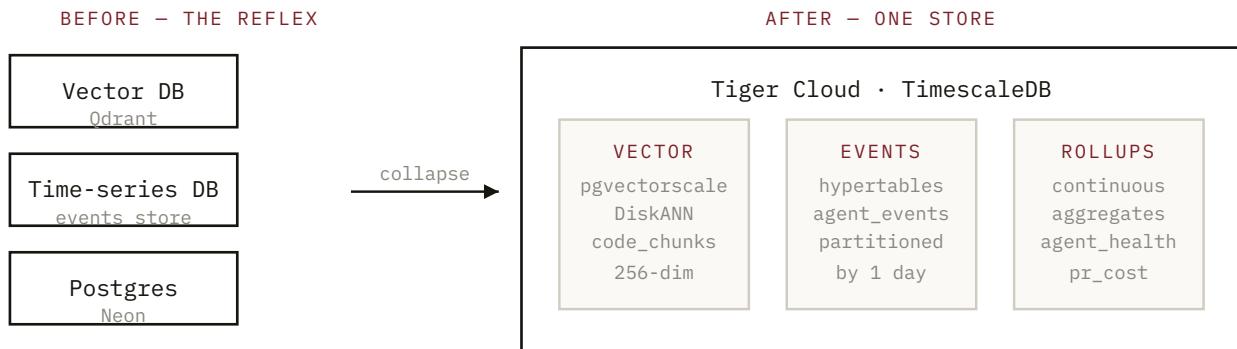
```
09:00 security llm.call cost=0.018 latency=1200
09:01 quality llm.call cost=0.011 latency=900
09:01 tests llm.call cost=0.007 latency=700
```

```
continuous aggregate: agent_health_1m
```

```
09:00 security calls=1 cost=0.018 p95_ms=1200
09:01 quality calls=1 cost=0.011 p95_ms=900
09:01 tests calls=1 cost=0.007 p95_ms=700
```

The BudgetGuard can read the summary first. If today's spend is already above the limit, it blocks before the next LLM call happens.

That gives us one store with three lanes inside it. The product stays easier to reason about: one durable database, one backup story, one place to query, and one PR identity that connects memory, truth, and time.



THE COLLAPSE. BEFORE: QDRANT FOR MEMORY, A DEDICATED TIME-SERIES STORE FOR EVENTS, AND POSTGRES FOR TRUTH. AFTER: TIGER CLOUD SERVES THE THREE SHAPES AS INTERNAL LANES IN ONE POSTGRES-COMPATIBLE DATABASE.

Redis stays — judgment, not dogma

The job queue is still Redis + ARQ. That is intentional. Queue data is short-lived and high-churn; it does not need vector search, time buckets, or dashboard rollups. "One database" here means one durable data spine, not forcing every workload into SQL.

Create the Tiger Cloud account

For the TigerData implementation path, create a Tiger Cloud account before the coding phase. Go to the Tiger Cloud signup page, start a free trial, and create a new account. New accounts can receive **\$1,000 in credit, valid for 30 days, with no credit card required**. The credit is for new accounts only.

1. Open the Tiger Cloud signup page and choose **Sign up for Tiger Cloud**.
2. Enter full name, work email, and a password of at least 12 characters.
3. Create the Tiger Cloud service, then copy the Postgres connection string.

4. Keep the connection string and passwords in the backend `.env` file, not in source code.

CREDENTIALS AND VALUES TO SAVE FOR LATER

VALUE	WHERE IT COMES FROM	WHERE WE USE IT
TIGER_DATABASE_URL	Tiger Cloud connection string	Backend database connection and TigerMemoryClient
Database password	Tiger Cloud service credentials	Part of the connection string; keep it in <code>.env</code> , never in HTML or Git
OPENAI_API_KEY	OpenAI dashboard	Embeddings and LLM calls
GitHub App credentials	GitHub developer settings	Webhook verification and posting PR reviews

The Tiger value should look like a Postgres URL, usually with SSL enabled. In the app, keep it as an environment variable, for example

`TIGER_DATABASE_URL=postgres://... .` The code should read it from settings; it should not be pasted into source files.

```
# backend/.env
TIGER_DATABASE_URL=postgres://USER:PASSWORD@HOST:5432/DB?sslmode=require
OPENAI_API_KEY=sk-...
GITHUB_APP_ID=...
GITHUB_WEBHOOK_SECRET=...
GITHUB_PRIVATE_KEY_PATH=...
```

CARRY INTO THE ARCHITECTURE

One Postgres-compatible data spine. Tiger Cloud carries memory with pgvector/pgvectorscale, time with hypertables and continuous aggregates, and truth with normal relational tables. Redis stays for the queue.

2.3 The Three Lanes, in Real Schema

"One database" does not mean one giant table. It means the same database holds different table shapes for different jobs.

Lane 1 – Memory: **code_chunks**

This replaces the old Qdrant collection. The ingestion job chunks repository files, embeds each chunk, and writes it here. At review time,

`backend/memory/tiger_client.py` searches this table to find code similar to the PR diff.

```
CREATE TABLE IF NOT EXISTS code_chunks (  
    id            UUID            PRIMARY KEY DEFAULT gen_random_uuid(),  
    repo          TEXT            NOT NULL,  
    path          TEXT            NOT NULL,  
    symbol        TEXT,           -- function/class name (nullable)  
    chunk_index   INT             NOT NULL,   -- order within file  
    content       TEXT            NOT NULL,  
    embedding     VECTOR(256)     NOT NULL,   -- text-embedding-3-large, 256 dims  
    token_count   INT,  
    updated_at    TIMESTAMPTZ     NOT NULL DEFAULT now()  
);  
  
CREATE INDEX IF NOT EXISTS code_chunks_emb_idx  
    ON code_chunks USING diskann (embedding vector_cosine_ops);  
  
ALTER TABLE code_chunks  
    ADD COLUMN IF NOT EXISTS content_tsv TSVECTOR  
        GENERATED ALWAYS AS (to_tsvector('english', content)) STORED;  
  
CREATE INDEX IF NOT EXISTS code_chunks_fts_idx  
    ON code_chunks USING GIN (content_tsv);
```

The key idea: vector search catches meaning; full-text search catches exact names like function names, error codes, and config keys. The code uses both and merges the results.

Lane 2 – Time: **agent_events**

This replaces the instinct to add a separate logs or time-series database. Every agent action becomes one append-only row: span starts, span ends, LLM calls, tool calls,

decisions, escalations, cost, latency, and payload. The code path is `backend/observability/events.py`.

```
CREATE TABLE IF NOT EXISTS agent_events (
  ts          TIMESTAMPTZ NOT NULL,
  review_id   UUID        NOT NULL,
  agent       TEXT        NOT NULL, -- security|quality|tests|docs|aggrega
  span_id     UUID        NOT NULL DEFAULT gen_random_uuid(),
  parent_span UUID,
  event_type  TEXT        NOT NULL, -- span.start|span.end|llm.call|tool.c
                                   -- |decision|escalation

  model       TEXT,
  tokens_in   INT,
  tokens_out  INT,
  cost_usd    NUMERIC(10,6),
  latency_ms  INT,
  outcome     TEXT,          -- approved|request_changes|critical_block|escal
  confidence  NUMERIC(4,3),
  payload     JSONB
);

SELECT create_hypertable(
  'agent_events',
  by_range('ts', INTERVAL '1 day'),
  if_not_exists => TRUE
);
```

Because this table is a hypertable, the database understands that time is the natural partition key. Old data can be compressed or retained differently from fresh data, and recent queries stay narrow.

Lane 3 – Rollups: dashboard-ready views

The dashboard does not want raw events. It wants answers: cost per agent, p95 latency, token usage, and per-PR cost. Continuous aggregates precompute those answers from `agent_events`. The code reads them in `backend/economics/cost_repository.py`.

```
CREATE MATERIALIZED VIEW IF NOT EXISTS agent_health_1m
WITH (timescaledb.continuous) AS
SELECT
```

```

        time_bucket('1 minute', ts)          AS bucket,
        agent,
        count(*) FILTER (WHERE event_type = 'llm.call')    AS llm_calls,
        sum(cost_usd)                                     AS cost_usd,
        approx_percentile(0.95, percentile_agg(latency_ms)) AS p95_ms,
        count(*) FILTER (WHERE outcome = 'rejected')::float
        / NULLIF(count(*) FILTER (WHERE outcome IS NOT NULL), 0) AS rejection_r
FROM agent_events
GROUP BY bucket, agent
WITH NO DATA;

SELECT add_continuous_aggregate_policy(
    'agent_health_1m',
    start_offset      => INTERVAL '2 hours',
    end_offset        => INTERVAL '1 minute',
    schedule_interval => INTERVAL '1 minute',
    if_not_exists     => TRUE
);

-- pr_cost_hourly: per-PR cost + token rollup, refreshed hourly
CREATE MATERIALIZED VIEW IF NOT EXISTS pr_cost_hourly
WITH (timescaledb.continuous) AS
SELECT
    time_bucket('1 hour', ts) AS bucket,
    review_id,
    sum(cost_usd)              AS total_cost_usd,
    count(DISTINCT agent)      AS agents_used,
    max(confidence)            AS max_confidence
FROM agent_events
GROUP BY bucket, review_id
WITH NO DATA;

```

The relational tables – same database, same transaction

The truth lane is deliberately ordinary: `pr_review_records` for one row per review, `finding_records` for one row per finding, `hitl_reviews` for human review state, and `hitl_feedback` for human feedback. These are normal relational tables because the shape is normal relational data.

The simple mental model: memory is `code_chunks`, truth is the review tables, and time is `agent_events`. One PR ties them together.

2.4 Why Tiger Cloud Beats the Split

The point is not that specialized databases are bad. The point is that this product gets simpler when the durable data lives in one Postgres-compatible place.

Tiger Cloud vs plain Postgres

Plain Postgres is enough for review rows and findings. It is not enough, by itself, for this whole agent. We also need fast vector search over code embeddings and efficient time-series queries over millions of agent events. Tiger Cloud keeps the Postgres programming model but adds TimescaleDB, pgvector, and pgvector scale.

Tiger Cloud vs Qdrant

Qdrant is good at vector search. The problem is that vector search is not the only question this agent asks. A PR review needs vector similarity plus repo filters, freshness, exact identifier matching, review records, cost records, and audit history. Keeping vectors in Tiger means the retrieval result can live beside the metadata and the review trail.

Why DiskANN matters

The simple version: embeddings are lists of numbers, and vector search asks, "which stored lists are closest to this new list?" Many vector indexes keep a lot of their search graph in memory. That gets expensive as the number of code chunks grows. DiskANN is designed to keep more of the index on SSD while still returning close neighbors quickly. That is why pgvector scale can make Postgres realistic for large vector memory, not just toy demos.

Why hypertables matter

Agent events are naturally time-ordered. The dashboard usually asks recent-time questions: last hour, last day, last week. A hypertable partitions the event stream by time, so recent queries touch recent chunks instead of treating the table like one endless pile.

Why continuous aggregates matter

The app should not calculate "daily spend" by scanning raw LLM calls every time someone opens the dashboard. Continuous aggregates keep summary tables warm: cost per agent, latency percentiles, token usage, and per-PR cost. The dashboard reads the summary; Tiger keeps it updated.

ALTERNATIVES CONSIDERED AND REJECTED (ADR-003)

OPTION	REJECTED BECAUSE
Keep Qdrant + Postgres	Works, but splits memory from review truth and audit history.
Plain Postgres only	Good for truth, weak for large vector memory and time-series rollups.
Add ClickHouse for events	Powerful, but another durable store, connection, schema, and failure mode.
Add Jaeger or Tempo for traces	Useful for infra tracing, but now the product audit trail is outside the product database.

TRADE-OFFS ACCEPTED

TRADE-OFF	RATIONALE
Tiger Cloud is a managed service	Accepted because it replaces multiple durable stores and keeps the system easier to operate.
There are two access styles	SQLAlchemy stays for normal relational work; asyncpg is used for hot paths like event inserts and chunk upserts.
Redis still exists	Accepted because Redis is a queue/cache, not the durable data spine.

THE QUESTION WE ASK

When I choose one database, am I simplifying the product, or am I hiding a workload that the database cannot actually handle?

Generalizes to: consolidation is good only when the single store handles each data shape honestly. Name the shapes, name the missing capabilities, and check that the chosen store really has them.

CARRY INTO THE ARCHITECTURE

A simpler single spine. Tiger Cloud wins here because it keeps memory, truth, and time in one Postgres-compatible store while still giving each lane the index or table shape it needs.

PART III / THE ARCHITECTURE, ASSEMBLED

Now, and only now, the boxes

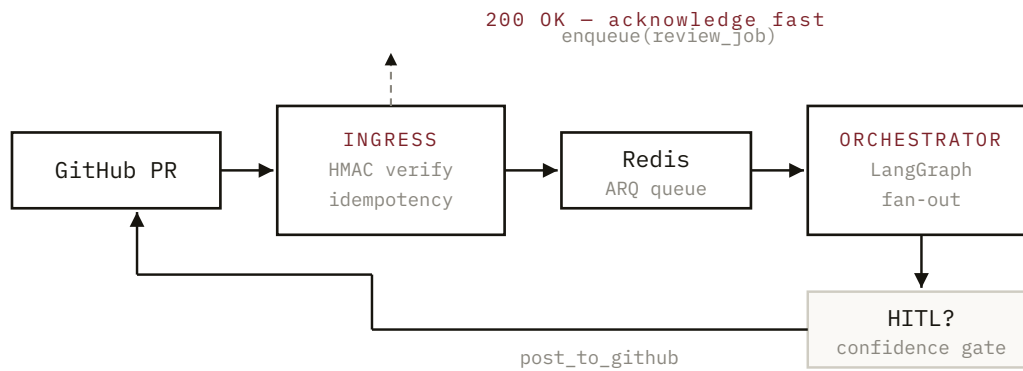
Every box below was earned in Parts I and II. As each section opens, it names the principle it fell out of. The diagrams are the carried pieces, drawn. Nothing here is new – it is the reasoning model made concrete.

3.1 Ingress & the Queue

From the ingress trigger and the decoupling we carried at L2 and L8: a GitHub webhook arrives, is validated, and is enqueued – never handled inline.

The ingress handler does exactly three things. It verifies the GitHub **HMAC-SHA256 signature** on the payload, rejecting forgeries before any work. It checks the **idempotency key** (the `X-GitHub-Delivery` UUID) so a retried delivery is acknowledged and dropped rather than reviewed twice – the idempotency defense from L8. Then it **enqueues** the job to Redis + ARQ and returns 200 immediately. GitHub expects a fast acknowledgment; the heavy work happens asynchronously in an ARQ worker.

The queue decouples ingress from review. A slow LLM provider or a crashed orchestrator can never make the webhook endpoint time out. This is why the work is enqueued and not done in the request – correctness, not just performance.



THE PR LIFECYCLE. INGRESS VALIDATES AND ENQUEUES; THE ARQ WORKER DRIVES THE ORCHESTRATOR; THE RESULT RETURNS THROUGH THE CONFIDENCE GATE TO GITHUB.

INTERROGATING QUESTION

Where does this break at 10 000 PRs per minute?

Queue depth outgrows worker drain rate; the single ARQ worker becomes the bottleneck. The modular-monolith answer (ADR-002): extract the webhook receiver as a stateless ingress service and the orchestrator as a separate worker pool – when the trigger is measured, not anticipated.

3.2 The Orchestrator

From the parallel-specialists fan-out we carried at L3: the orchestrator is a graph, not a pipeline. Nodes run simultaneously; state is checkpointed between them.

The orchestration engine is **LangGraph**. It defines the workflow as a directed graph of nodes (functions or LLM calls) and edges (what runs next). Parallel fan-out – running the four specialists at once – is first-class via the **Send API**. A typed state object flows through the graph, and LangGraph **checkpoints that state to Redis at each node boundary**, so a worker crash mid-review resumes from the last completed node rather than restarting. The checkpoint store is the same Redis the queue uses – one fewer dependency.

The graph lives in `backend/orchestrator/ : graph.py` wires the `StateGraph` and the `Send` fan-out, `state.py` defines the typed state, `nodes.py` holds the node functions (`build_context` , the four specialist nodes, `aggregate`), and `langgraph_engine.py` implements the engine interface. The aggregator node is

wired to run only after all four specialist nodes complete. The graph encodes that join; we do not orchestrate it by hand. This is the orchestration-deadlock defense from L8: every node has a timeout, and the join cannot hang forever on a single stalled agent.

3.3 Trade-off: LangGraph vs Temporal

The orchestrator had two realistic candidates. The decision (ADR-001) turns on a clear need and the discipline that keeps it reversible.

The need is threefold: coordinate four parallel sub-agents, persist workflow state across steps so a crash does not lose work, and handle retries cleanly when an LLM or tool call fails.

THE TWO CANDIDATES (ADR-001)

	LANGGRAPH (CHOSEN)	TEMPORAL
Where it runs	Inside our Python process – zero extra infrastructure	A separate server plus separate worker processes
Parallel fan-out	First-class via the Send API – exactly what four agents need	Supported, but heavier to express
Checkpointing	To the same Redis we already run for the queue	Durable, built-in, very strong guarantees
LLM integration	Tight tool-calling integration; fast local iteration	Generic; not LLM-specific
Maturity / scale	Newer; unproven at thousands of concurrent workflows	Battle-hardened (Uber, Netflix); excellent at scale
Operational cost	None beyond the app	Meaningful ops overhead before we understand our workflow shapes

The decision: use LangGraph for Phases 1–12. The discipline that makes this safe is a single abstract interface, `backend/core/workflow_engine.py`, with three methods – `run(workflow_id, input)`, `resume(workflow_id, state)`,

`get_state(workflow_id)` . The LangGraph implementation lives in `backend/orchestrator/langgraph_engine.py` . All orchestrator code imports from `core.workflow_engine` , never from LangGraph directly. If scale demands Temporal at Phase 13+, we write a Temporal implementation of the same interface and swap it in – nothing else in the codebase changes.

This is the "defer the expensive decision, keep the door open" principle: make the cheaper choice now, and pay for Temporal only when scale actually demands it. **Revisit** if sustained concurrent workflows exceed 50 per minute, if cross-service coordination is needed, or if Redis checkpointing proves insufficient against data loss.

THE QUESTION WE ASK

Can I make the cheaper decision now and hide the harder one behind an interface, so swapping it later changes one file, not the system?

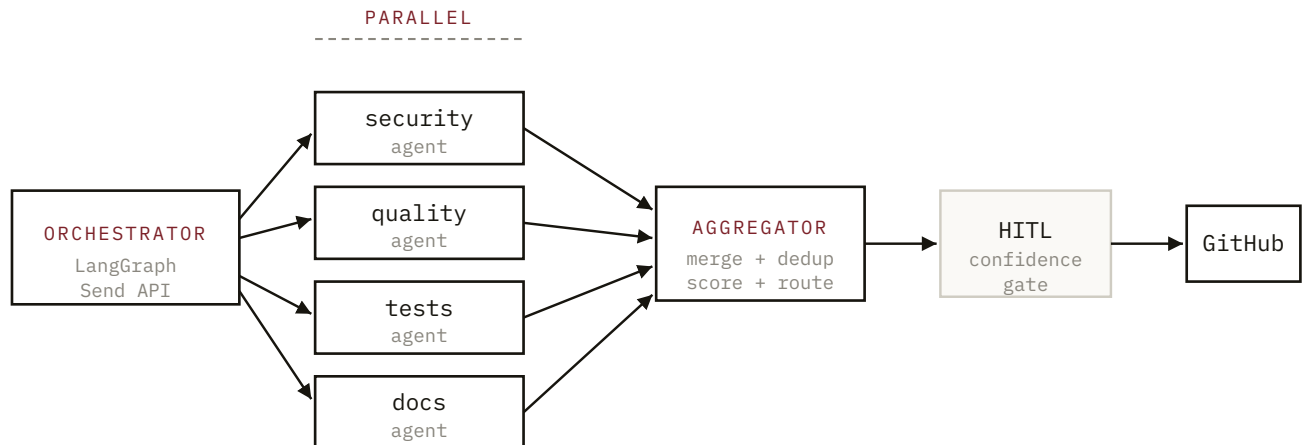
Generalizes to: any decision you are unsure about. Put it behind a narrow interface, pick the simple implementation, and let the seam be where the future swap happens. Premature commitment to the "scalable" option is its own kind of debt.

3.4 Specialists & the Aggregator

From the four concerns we carried at L1 and the Finding contract from L2: four specialists run in parallel, each returning structured findings, merged by one aggregator.

The four specialists – `security` , `quality` , `tests` , `docs` – share a base shape in `backend/agents/base_agent.py` (BudgetGuard check, retrieval call, LLM call, event emission, error handling) and differ only in their domain prompt and post-processing. Each returns a list of **Finding** objects (defined in `agents/contracts.py`) matching the L2 contract: `agent_type` , `severity` , `category` , `summary` , `file_path` , `line_start / line_end` , `suggestion` , `confidence` , `rationale` . Structured output is what makes the next step deterministic – the aggregator merges *data*, not prose.

The aggregator merges all four lists, deduplicates findings that multiple agents raised on the same file and line (keeping the highest-confidence one and noting the agreement), computes an `overall_confidence`, and applies the L7 HITL gate: post automatically when confident and free of CRITICAL findings, otherwise insert into the human approval queue.

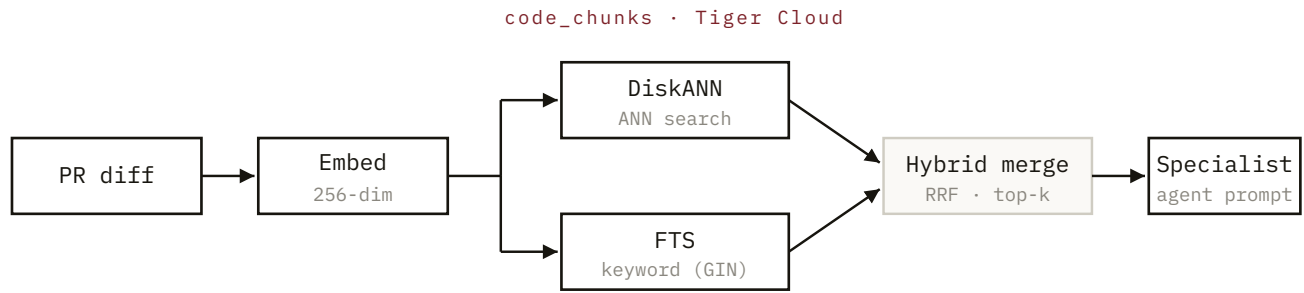


FAN-OUT VIA LANGGRAPH SEND API. ALL FOUR SPECIALISTS RUN SIMULTANEOUSLY. THE AGGREGATOR MERGES, DEDUPLICATES, SCORES, AND ROUTES THROUGH THE HITL GATE BEFORE POSTING TO GITHUB.

3.5 The Retrieval Path

From the grounding problem at L4 and the data shapes at L5: each specialist queries the vector lane for codebase context relevant to the diff. Retrieval is hybrid – vector and keyword in parallel.

The retrieval layer is `backend/memory/ : tiger_client.py` (the `TigerMemoryClient`), `embedder.py` (text-embedding-3-large, 256-dim), and `context_retriever.py` (hybrid merge). Pure vector search finds meaning but misses exact identifiers; pure keyword search finds exact strings but misses semantic relevance. The layer runs both against `code_chunks : DiskANN ANN search` over the 256-dim embeddings, and **full-text search** over the `content_tsv GIN` index. A hybrid merge fuses the two result sets by reciprocal rank and returns the top-k chunks into the specialist's prompt. The `repo_file_index` table tracks freshness so the ingestion pipeline only re-embeds files that changed.



RETRIEVAL PATH. THE DIFF IS EMBEDDED AND QUERIED AGAINST BOTH THE DISKANN VECTOR INDEX AND THE FTS GIN INDEX – THE TWO LANES OF ONE TABLE – MERGED BY RECIPROCAL RANK FUSION INTO THE SPECIALIST PROMPT.

INTERROGATING QUESTION

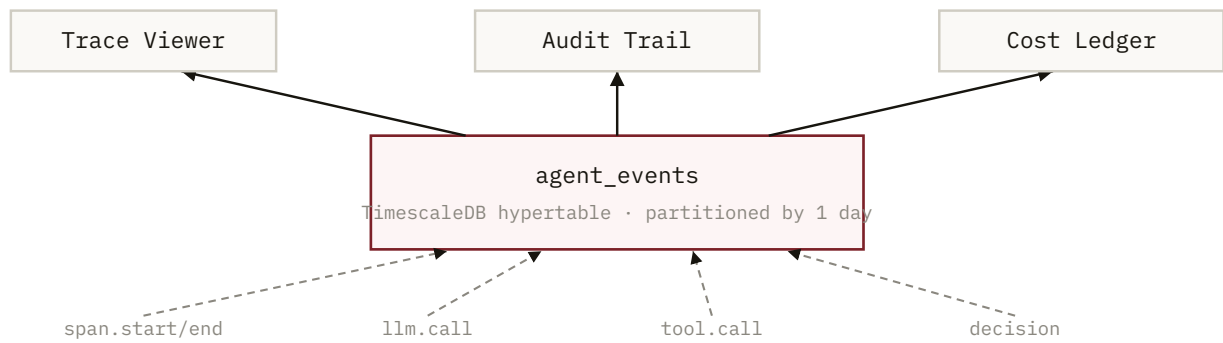
What happens when the embeddings go stale – a function is refactored but its chunk still describes the old version?

The `repo_file_index.last_indexed_at` drives incremental re-embedding; the `code_chunks_unique_idx` on `(repo, path, chunk_index)` lets the upsert overwrite stale chunks. The real question is whether a weekly full reindex is cheaper than on-demand freshness – it depends on repo churn.

3.6 The Events Spine in Operation

From the trust-and-proof principle at L6: every action is one row in `agent_events`, and three consumers read that one table.

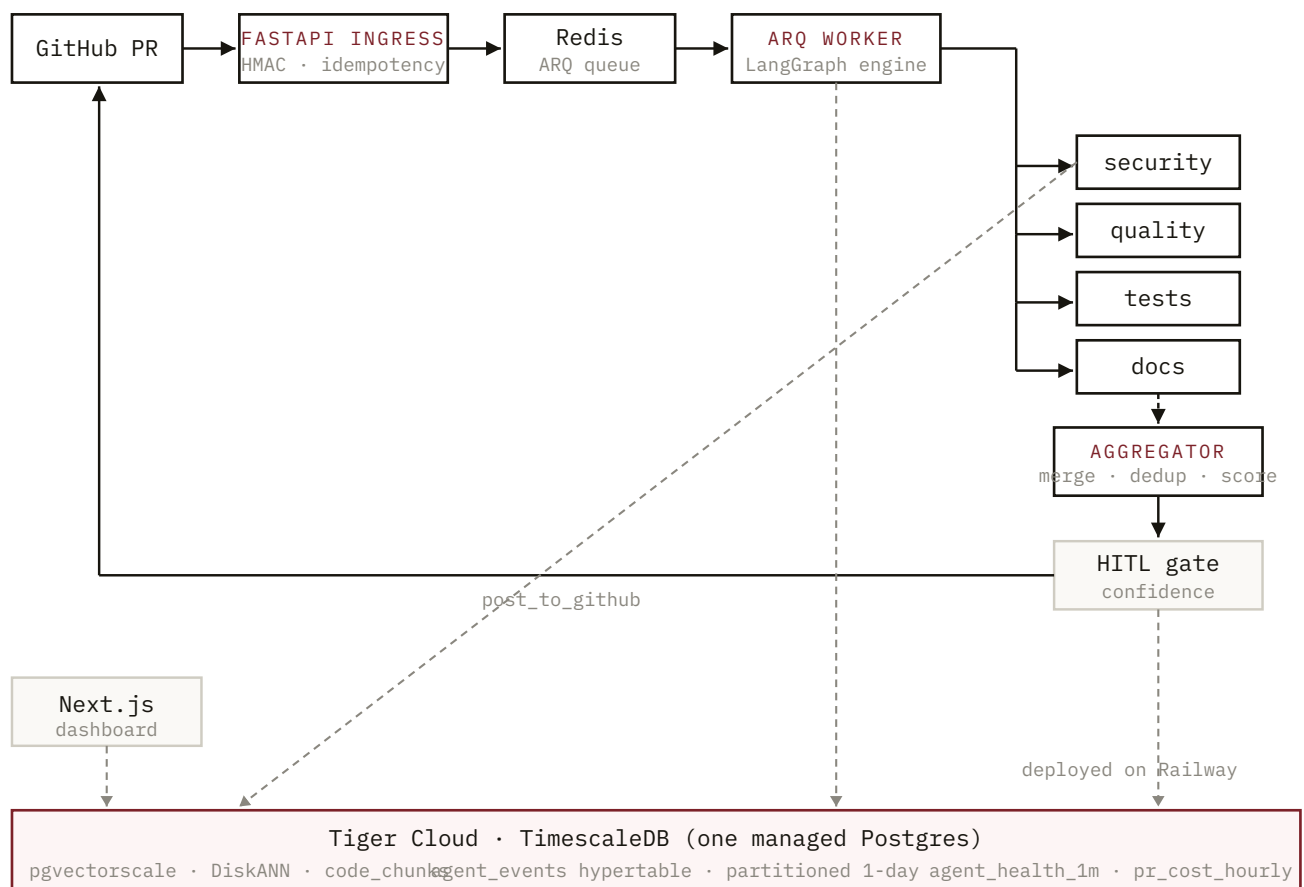
The observability layer (`backend/observability/: events.py, tracing.py, audit.py`) emits an event for every span, LLM call, tool call, and decision, carrying the `span_id / parent_span` chain, cost, latency, confidence, and outcome. The **trace viewer** reconstructs any review with `SELECT ... WHERE review_id = $1 ORDER BY ts`. The **audit trail** is the same append-only table, immutable by construction. The **cost ledger** reads the continuous aggregates – and so does the BudgetGuard, which reads the day's running cost from `agent_health_1m` at the top of every agent run and hard-blocks before any LLM call if the daily cap is exceeded (ADR-004). Continuous aggregates also surface drift: a rising `rejection_rate` per agent is the calibration signal for continuous learning.



THE AGENT_EVENTS HYPERTABLE SERVES THREE CONSUMERS FROM ONE TABLE – THE TRACE VIEWER, THE AUDIT TRAIL, AND THE COST LEDGER ARE THREE QUERIES AGAINST ONE TIME-ORDERED SPINE.

3.7 The Full System

Every carried piece, in one picture. GitHub to ingress to queue to the four-agent fan-out to the aggregator and HITL gate, with the Tiger Cloud spine beneath all of it and the Next.js dashboard reading the aggregates.



THE FULL SYSTEM. GITHUB → FASTAPI INGRESS → REDIS/ARQ → LANGGRAPH (FOUR PARALLEL AGENTS) → AGGREGATOR → HITL GATE → GITHUB. TIGER CLOUD IS THE SHARED SPINE BENEATH EVERY COMPONENT; THE NEXT.JS DASHBOARD READS THE CONTINUOUS AGGREGATES.

PART III – CARRY FORWARD

The reasoning model, now drawn:

- Ingress (HMAC + idempotency) and Redis/ARQ decoupling – from L2, L8.
- LangGraph fan-out with Redis checkpointing, abstracted behind `core/workflow_engine.py` – from L3, ADR-001.
- Four specialists returning `Finding` s, merged and routed by the aggregator through the confidence gate – from L1, L2, L7.
- Hybrid DiskANN + FTS retrieval and the `agent_events` spine – from L4, L5, L6, all on the one Tiger Cloud store from Part II.

PART IV / THE IMPLEMENTATION PLAN

From design to code

The architecture is derived; now the build. This part enumerates everything that gets built, in what order, and how each step proves itself green before the next begins. No code is shown – this is the inventory and the sequence, so the full scope is visible before a line is written.

4.1 The 20-Phase Build Roadmap

The build follows the 0.4 lifecycle exactly. Each phase proves one thing, ends green, and has a written gate before the next starts. Tiger Cloud is load-bearing in five phases, marked below.

THE BUILD ROADMAP – PHASE, WHAT IT PROVES, ITS GATE

#	PHASE	WHAT IT PROVES / ITS GREEN GATE	TIGER
0	Cognitive Design	Autonomy level and HITL boundaries are decided and written	

#	PHASE	WHAT IT PROVES / ITS GREEN GATE	TIGER
1	System Architecture	Module graph and ADRs exist; dependency rule defined	
2	Frontend Engineering	Dashboard shell renders; streaming wired	
3	Backend & API	FastAPI up; webhook validates HMAC; idempotency holds	
4	Workflow Orchestration	LangGraph fans out to 4 nodes in parallel; checkpoints resume	
5	LLM & Reasoning	Model routing per agent; prompt registry versioned	
6	Memory Architecture	RAG on pgvectorscale; hybrid retrieval returns top-k	Tiger
7	Tooling & Sandboxing	Tool registry enforces scope; Docker sandbox isolates	
8	Multi-Agent Systems	4 specialists + contracts + aggregator produce one review	
9	Evaluation	Golden dataset runs; LLM-as-judge scores; regression gate blocks	
10	Observability & Tracing	OTel spans land in the agent_events hypertable	Tiger
11	Security	Threat model written; RBAC enforced; audit trail immutable	
12	Reliability	Retries, circuit breakers, idempotency verified under fault injection	
13	Infrastructure	Tiger Cloud provisioned; Tiger MCP wired	Tiger
14	Data Engineering	Ingestion pipeline runs; hypertable schema designed and migrated	Tiger
15	Governance	Audit logs queryable; explainability per finding	
16	Economics & Cost Control	Per-agent cost via continuous aggregates; BudgetGuard hard-blocks	Tiger
17	Developer Experience	Prompt playground and trace viewer usable	

#	PHASE	WHAT IT PROVES / ITS GREEN GATE	TIGER
18	CI/CD for AI	Prompt versioning; eval gates; canary release path	
19	Human-in-the-Loop	Approval queue, escalation, dispute, feedback all wired	
20	Continuous Learning	Drift detection reads continuous aggregates	Tiger

4.2 The Module Map

This is the exact code we will build, in modular-monolith form: one process, 23 internal modules, inward-only dependencies (ADR-002). Below is the full surface area – every module and its files – so the scope is visible before the first commit.

THE 23 MODULES OF THE MONOLITH, PLUS MIGRATIONS AND FRONTEND

MODULE	FILES	PURPOSE
agents/	base_agent, contracts, security_agent, quality_agent, test_agent, docs_agent	The four specialists and their shared base + Finding contract
api/	reviews, economics_router, hitl_router, queue, schemas	REST endpoints for reviews, economics, HITL, queue status
auth/	dependencies	RBAC dependencies for FastAPI routes
core/	workflow_engine, exceptions	The abstract orchestration interface and shared exception types
data/	ingestion, freshness	Code-chunk ingestion pipeline and re-embed freshness tracking
database/	postgres, models, repository	Async engine + Tiger pool + init_tiger_schema; ORM models; repos
economics/	cost_repository, budget, routing_advisor	Reads aggregate views; BudgetGuard; model-routing advice

MODULE	FILES	PURPOSE
evaluation/	golden_dataset, judge, regression_gate	Golden PRs, LLM-as-judge, regression gate for CI
hitl/	queue, escalation, feedback, dispute	Approval queue, escalation engine, feedback capture, dispute API
integrations/	github_client, github_models	GitHub REST wrapper with retry; GitHub payload models
job_queue/	arq_worker	ARQ worker process consuming review jobs from Redis
memory/	tiger_client, embedder, context_retriever, redis_client	TigerMemoryClient (pgvectorscale + hybrid), embedding, retrieval, session cache. qdrant_client retired per ADR-003
models/	enums, findings, review, webhook	Pydantic schemas: Finding, Review, WebhookEvent, enums
observability/	events, tracing, audit, alerting, logging, workflow_context	emit_agent_event → hypertable; OTel; audit; alerts; ContextVar
orchestrator/	graph, nodes, state, langgraph_engine	LangGraph StateGraph, node functions, typed state, engine impl
prompts/	registry, templates/	Prompt registry + versioned prompt files per agent
reliability/	retry, circuit_breaker, idempotency, timeout	The L8 reliability mechanics
security/	threat_model, injection_guard, rbac, masking	Threat model, prompt-injection guard, RBAC, secret masking
tools/	tool_registry, model_router, llm_client, sandbox, capability_scope	Tool catalog, model routing, LLM client, Docker sandbox, scoping

MODULE	FILES	PURPOSE
webhook_receiver/	validator, parser, router	HMAC validation, payload parsing, event routing to the queue
migrations	scripts/migrations/2026-06-tiger-init.sql	Idempotent schema DDL – the lanes and tables from Part II
frontend/	src/app, components, lib	Next.js: review dashboard, HITL queue, trace viewer, economics page reading continuous aggregates

READ THIS AS A CONTRACT

This is the exact code we will build, in this order, each phase ending green. The dependency rule from ADR-002 holds throughout: `core` depends on nothing; outer modules depend inward only; `observability` is cross-cutting, injected as middleware. You can delete any outer module and the inner ones still compile.

4.3 The Tiger Integration Plan

The migration onto the single data spine is staged in four phases (ADR-003), each independently verifiable.

THE FOUR-PHASE TIGER INTEGRATION (ADR-003)

PHASE	WHAT HAPPENS	VERIFIED BY
A · Infra	Provision Tiger Cloud, run <code>2026-06-tiger-init.sql</code> , verify extensions (<code>timescaledb</code> , <code>vector</code> , <code>vectorscale</code>)	Extensions present; hypertable and aggregates listed
B · Events	Wire <code>emit_agent_event()</code> in the orchestrator nodes and the LLM client	Every span, <code>llm.call</code> , <code>tool.call</code> lands in <code>agent_events</code>
C · Memory	Retire <code>qdrant_client.py</code> ; test hybrid retrieval end-to-end on <code>code_chunks</code>	DiskANN + FTS return top-k; recall verified
D · Dashboard	Wire the continuous-aggregate endpoints to the frontend economics page	Per-agent cost and p95 latency render from <code>agent_health_1m</code>

The migration is greenfield — there is no live data to move — so it does not need to be zero-downtime. Each phase leaves the system green before the next begins, the same discipline the 20-phase roadmap uses.

CLOSING

How to reuse this

The architecture was the example. The method is the takeaway. The questions we asked at each level are portable; the next system you have never seen can start from them instead of from a blank page.

5.1 The Transferable Framework

Collect the "question we ask" moves from the climb. None of them mention code review. Each applies to whatever you build next.

1. **Why does this system exist?** Name the one scarcity it relieves, or do not build it.
2. **What mature system already solved a version of this?** Borrow its decomposition.
3. **What is the precise trigger, output, and the object on the arrows?** The contract matters more than the boxes.
4. **What does the mature version decompose into?** Find the hidden back half; stand on the highest rung you can afford.
5. **What does this reasoner need that is not in front of it?** Retrieve exactly that, and only that.
6. **What distinct kinds of state, by access pattern?** Kinds drive shapes; shapes drive stores.
7. **Can it prove how it got there, and what it cost?** Design the audit stream with the decision, not after.

8. **Where on the autonomy spectrum, and what signal moves a case along it?** Let the system earn trust case by case.
9. **What happens when each box fails?** Degrade to slower-but-correct, never fast-but-wrong.
10. **Count the data shapes before the databases.** One store until a shape proves it cannot live there – and prove the consolidation wins on the facts.
11. **Can the hard decision hide behind an interface?** Make the cheap choice now; swap one file later.

Notice that the most consequential architectural decision in this study – collapsing memory, truth, and time onto one Tiger Cloud spine – fell out of question ten, not out of a preference for any product. The store was derived, lane by lane, because the data shapes fit one Postgres-compatible database with the right extensions. That is the posture to carry: let the data shapes, the failure modes, and the proof requirement choose the architecture, and defend each choice on its merits.

Design is not a parts list. It is a chain of questions, each one earning the next box. If a component cannot be traced back to a question you can defend, it is either missing a justification or it does not belong.

FOR WORKING PROFESSIONALS

Want to truly learn AI-native engineering?

If you are a working professional and want to build the kind of proof needed for Applied AI Engineer, Forward Deployed Engineer, Agent Engineer, AI SWE, AI Product Engineer, or AI startup engineering roles, apply to work with us.

The AI-Native Engineering Sprint is for people who want to learn the systems behind agents, evaluation, retrieval, AI coding workflows, production capstones,

and technical positioning.

APPLY TO WORK WITH US

A first-principles architecture study · ADR-001 (LangGraph vs Temporal) · ADR-002 (modular monolith) · ADR-003 (Tiger Cloud data layer) · ADR-004 (cost control) · Schema: `scripts/migrations/2026-06-tiger-init.sql`

Grounded in the `ai-pr-review-agent` codebase · Built by Ayush Singh